

ENIAD TD3

POO en Java

Exercice 0 :

1. Qu'est-ce que l'héritage en programmation orientée objet et comment est-il implémenté en Java?
2. Comment déclarez-vous une classe héritée (ou sous-classe) en Java? Donnez un exemple.
3. Quelle est la différence entre l'héritage de classe (extends) et l'implémentation d'interface (implements) en Java?
4. Expliquez le concept de surcharge de méthodes dans le contexte de l'héritage en Java.
5. Quand devriez-vous utiliser une classe abstraite plutôt qu'une interface en Java? Donnez des exemples pour illustrer votre réponse.
6. Comment déclarez-vous une classe abstraite en Java? Quelles sont les caractéristiques principales d'une classe abstraite?
7. Peut-on instancier une classe abstraite en Java? Pourquoi ou pourquoi pas?
8. Quelle est la signification de l'interface en Java et comment déclarez-vous une interface?
9. Pouvez-vous donner un exemple concret d'utilisation d'une interface multiple en Java? Comment cela est-il géré?
10. Comment Java prend-il en charge la résolution multiple d'héritage (ou "diamond problem") et pourquoi est-ce important dans le contexte des interfaces?

Exercices :

C'est quoi la sortie des programmes ci-dessous ?

```
interface Shape {  
    void draw();  
}  
  
class Circle implements Shape {  
    public void draw() {  
        System.out.println("Drawing a circle");  
    }  
}  
  
public class Exercice1 {  
    public static void main(String[] args) {  
        Shape shape = new Circle();  
        shape.draw();  
    }  
}
```

```
abstract class Animal {  
    abstract void makeSound();  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Bark");  
    }  
}  
  
public class Exercice2 {  
    public static void main(String[] args) {  
        Animal dog = new Dog();  
        dog.makeSound();  
    }  
}
```

```
interface Vehicle {
    void start();
}

abstract class Car implements Vehicle {
    void stop() {
        System.out.println("Car stopped");
    }
}

class Sedan extends Car {
    public void start() {
        System.out.println("Sedan started");
    }
}

public class Exercice3 {
    public static void main(String[] args) {
        Sedan sedan = new Sedan();
        sedan.start();
        sedan.stop();
    }
}
```

```
abstract class Fruit {
    abstract void eat();
}

class Apple extends Fruit {
    void eat() {
        System.out.println("Eating an apple");
    }
}

public class Exercice4 {
    public static void main(String[] args) {
        Fruit fruit = new Apple();
        fruit.eat();
    }
}
```

```
abstract class Shape {
    abstract void draw();
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a rectangle");
    }
}

public class Exercice8 {
    public static void main(String[] args) {
        Shape shape = new Rectangle();
        shape.draw();
    }
}
```

```
interface Sound {
    void makeSound();
}

class Guitar implements Sound {
    public void makeSound() {
        System.out.println("Strumming the guitar");
    }
}

public class Exercice2 {
    public static void main(String[] args) {
        Sound instrument = new Guitar();
        instrument.makeSound();
        instrument.play();
    }
}
```

```

interface Playable {
    void play();
}

class Guitar implements Playable {
    public void play() {
        System.out.println("Strumming the guitar");
    }
}

public class Exercice4 {
    public static void main(String[] args) {
        Playable instrument = new Playable();
        instrument.play();
    }
}

```

```

interface PowerSource {
    int getPower();
}

abstract class Device {
    abstract void turnOn();
}

class Laptop extends Device implements PowerSource {
    int getPower() {
        return 45;
    }

    void turnOn() {
        System.out.println("Laptop is turning on.");
    }
}

public class Exercice1 {
    public static void main(String[] args) {
        Laptop laptop = new Laptop();
        laptop.turnOn();
        System.out.println("Power: " + laptop.getPower() + " watts");
    }
}

```

```

interface Shape {
    void draw();
}

abstract class DrawingTool {
    abstract void sketch();
}

class Pen extends DrawingTool implements Shape {
    void sketch() {
        System.out.println("Sketching with a pen");
    }

    public void draw() {
        System.out.println("Drawing a shape with a pen");
    }
}

public class Exercice6 {
    public static void main(String[] args) {
        Shape pen = new Pen();
        ((DrawingTool) pen).sketch();
        pen.draw();
    }
}

```

```

abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a circle");
    }
}

public class CompilationErrorEx1 {
    public static void main(String[] args) {
        Shape shape = new Circle();
        shape.draw();
    }
}

```

```
interface Sound {  
    void makeSound();  
}  
  
class Guitar implements Sound {  
    public void playSound() {  
        System.out.println("Strumming the guitar");  
    }  
}  
  
public class CompilationErrorEx2 {  
    public static void main(String[] args) {  
        Sound instrument = new Guitar();  
        instrument.playSound();  
    }  
}
```